

Assignment-11

Name: Chiraag Aggarwal

Enrollment No. : S25CSEU0176

Batch:06

Q1. What is ring counter and Johnson counter?

Differentiate between Ring counter and Johnson counter.

Solution:

1. Ring Counter

In a standard Ring counter, the Qoutput of the last flip-flop is connected directly to the D (input) of the first flip-flop. To start the cycle, a single "1" (logic high) is typically preset into the first flip-flop while the rest are cleared. As clock pulses occur, this single "1" circulates around the ring.

- States: For n flip-flops, there are exactly n states.
- Decoding: No extra decoding gates are needed because only one flip-flop is high at a time.

2. Johnson Counter

The Johnson counter (also known as a Switch-Tail or Twisted Ring counter) connects the inverted output of the last flip-flop to the D input of the first. This "twist" causes the register to fill up with 1s and then fill up with 0s in a cycle.

- States: For n flip-flops, there are $2n$ states.
- Efficiency: It provides twice as many states as a Ring counter using the same number of flip-flops.

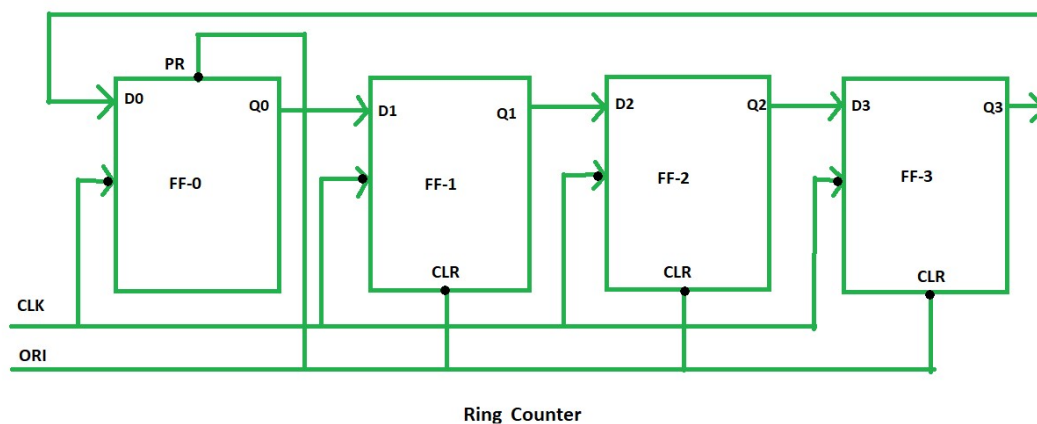
Q2. Design a 4-bit Ring Counter using D flip-flops. Write the Verilog code and show the output sequence and respective waveform.

Solution:

Circuit Design and Logic

In a 4-bit version, only one flip-flop is "set" (Logic 1) at any given time. As the clock pulses, this single HIGH bit circulates through the registers.

- Components: 4 D Flip-Flops, 1 Clock signal, 1 Reset/Preset signal.
- Initialization: Unlike other counters, a Ring Counter must be initialized with a specific pattern (usually 1000) using the Preset (PR) pin on the first FF and Clear (CLR) pins on the others.



State Table and Sequence

The sequence follows a pattern where the "1" moves one position to the right per clock cycle.

Clock Pulse	Q3	Q2	Q1	Q0
Reset	1	0	0	0
1st Pulse	0	1	0	0
2nd Pulse	0	0	1	0
3rd Pulse	0	0	0	1
4th Pulse	1	0	0	0

Verilog code

Testbench.SV

```
module tb_ring_counter;
    reg clk;
    reg reset;
    wire [3:0] q;

    ring_counter uut (
        .clk(clk),
        .reset(reset),
        .q(q)
    );

    always #5 clk = ~clk;

    initial begin

        clk = 0;
        reset = 1;

        #10 reset = 0;

        #100 $finish;
    end

    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(1, tb_ring_counter);
    end
end
```

```
endmodule
```

Design.SV

```
module ring_counter (  
    input clk,  
    input reset,  
    output reg [3:0] q  
);
```

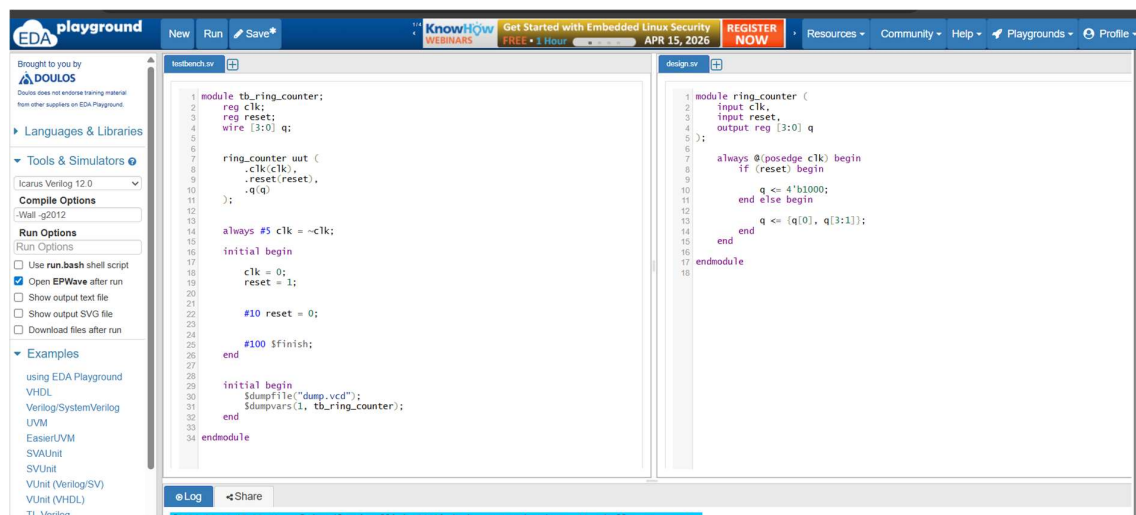
```
    always @(posedge clk) begin  
        if (reset) begin
```

```
            q <= 4'b1000;  
        end else begin
```

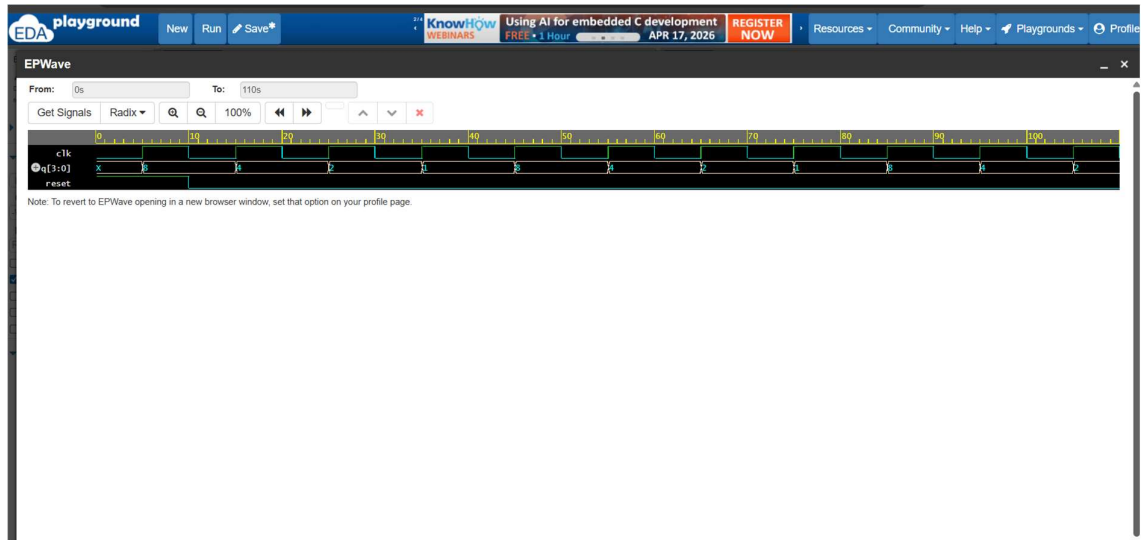
```
            q <= {q[0], q[3:1]};  
        end  
    end  
endmodule
```

```
endmodule
```

Screenshot of the code



Output



Q3. Design a 4-bit Johnson counter to generate the sequence:

0000 → 0001 → 0011 → 0111 → 1111 → 1110 → 1100 → 1000 → 0000

Write the Verilog code and verify the output. Also represent its waveform.

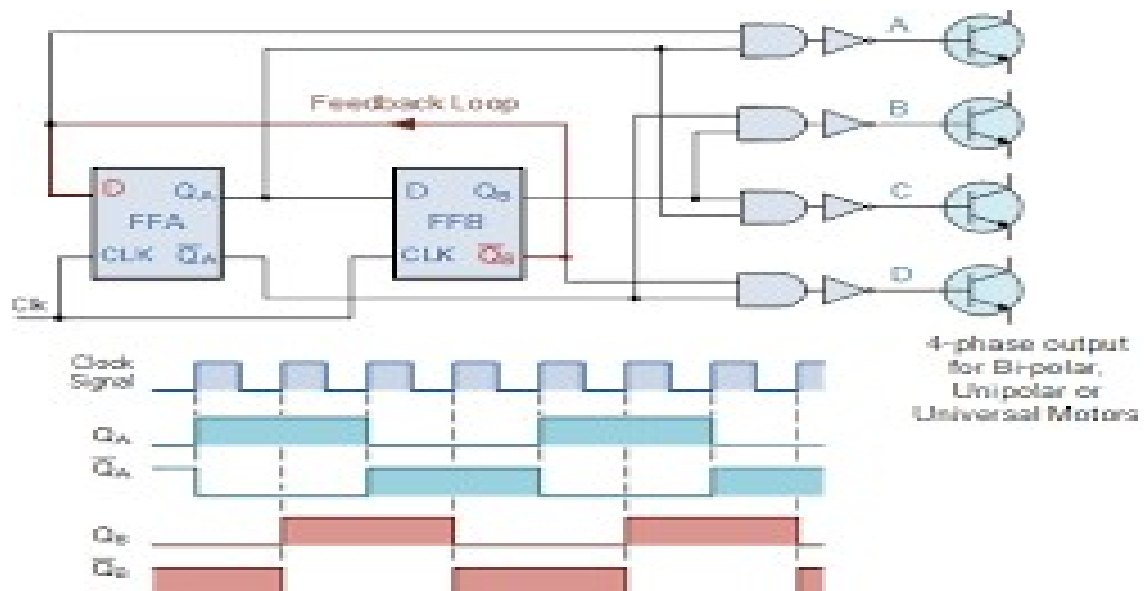
Solution:

Design Logic

To get the specific sequence you requested, the bits must shift from right to left, with the inverted feedback entering at the least significant bit (Q_0).

Logic: $D_0 = Q_3, D_1 = Q_0, D_2 = Q_1, D_3 = Q_2$.

- **Initialization:** A simple reset forces all flip-flops to 0000.



. Output Sequence & Waveform

The Johnson counter fills up with 1s and then "cleans" them out with 0s.

Clock Pulse	Q3	Q2	Q1	Q0	State
Reset	0	0	0	0	0
1	0	0	0	1	1
2	0	0	1	1	3
3	0	1	1	1	7
4	1	1	1	1	15
5	1	1	1	0	14
6	1	1	0	0	12
7	1	0	0	0	8
8	0	0	0	0	Back to 0

Verilog code

Testbench.SV

```
module tb_johnson_counter;
    reg clk;
    reg reset;
    wire [3:0] q;

    johnson_counter uut (
        .clk(clk),
        .reset(reset),
        .q(q)
    );

    always #5 clk = ~clk;

    initial begin
        clk = 0;
        reset = 1;
        #10 reset = 0;
        #100 $finish;
    end

    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(1, tb_johnson_counter);
    end

endmodule
```

Design.SV

```
module johnson_counter (
    input clk,
```

```

input reset,
output reg [3:0] q
);

```

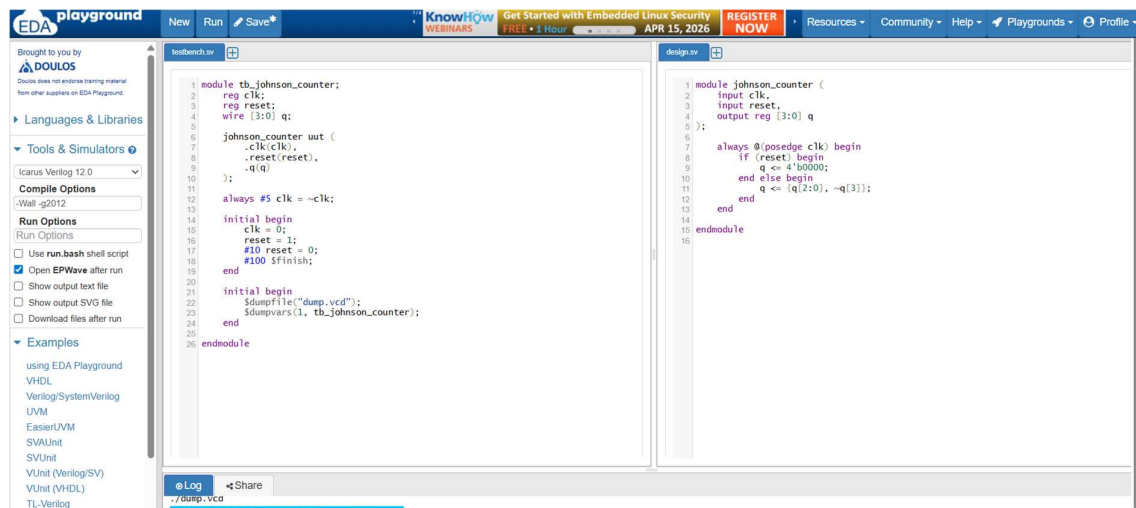
```

always @(posedge clk) begin
    if (reset) begin
        q <= 4'b0000;
    end else begin
        q <= {q[2:0], ~q[3]};
    end
end

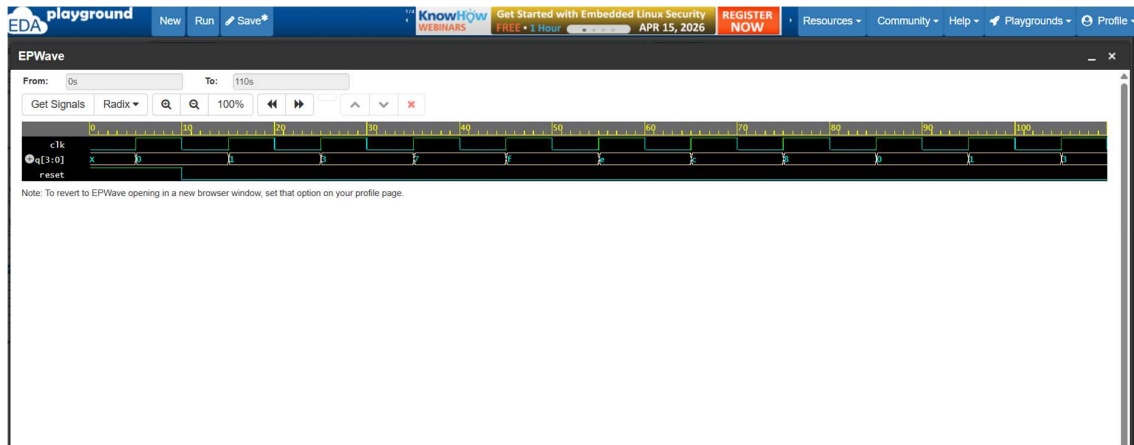
```

```
endmodule
```

Screenshot of the code



Output



Q4. Define a counter and its types. Explain the concept of a Mod-n counter with an example.

Solution:

A counter is a sequential logic circuit designed to count the number of clock pulses applied to its input. It is constructed using a series of flip-flops, where the state of the flip-flops represents the binary count.

Types of Counters

Counters are primarily classified based on how they are clocked:

1. Asynchronous (Ripple) Counter

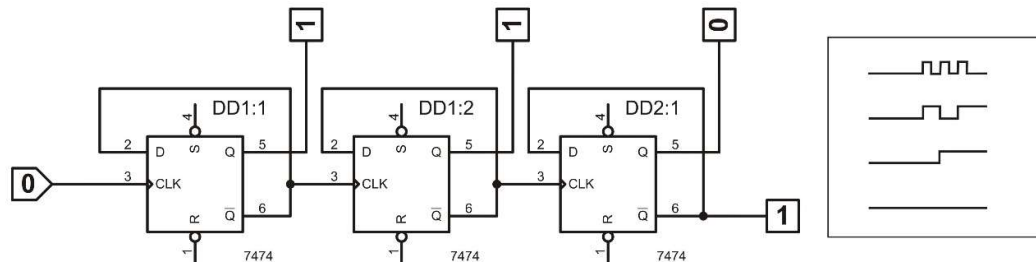
In this type, the clock signal is applied only to the first flip-flop. The output of the preceding flip-flop acts as the clock for the next one. This creates a "ripple" effect, leading to propagation delays.

2. Synchronous Counter

In this type, all flip-flops in the counter are triggered simultaneously by a common clock signal. This

eliminates propagation delay and is much faster than asynchronous counters.

A time diagram of the operation of a three-digit summing counter assembled from triggers. Graphs of input and output signals.



3. Other Classifications

- **Up Counter:** Counts from 0 to a maximum value.
- **Down Counter:** Counts from a maximum value down to 0.
- **Up/Down Counter:** Can count in either direction based on a control input.

Mod-n Counter

The Modulus (Mod) of a counter represents the total number of unique states the counter passes through before returning to its initial state.

- A counter with n flip-flops can have a maximum modulus of 2^n .
- A Mod- n counter uses logic gates (usually NAND or AND) to reset the counter once it reaches a specific count, allowing it to skip states.

Example: Mod-6 Counter

A Mod-6 counter counts through **6 unique states** (0, 1, 2, 3, 4, 5) and then resets to 0.

To design this, you need 3 flip-flops (since $2^2 < 6 < 2^3$). A standard 3-bit counter would normally count to 7 (111), but for a Mod-6 counter, we force a reset when the count reaches 6 (110 in binary).

State Sequence:

000 → 001 → 101 → 011 → 100 → 101 → (Resets on 110) → 000